

Do Transformers Learn Base-Agnostic Arithmetic?

A Mechanistic Interpretability Study of Cross-Surface Addition Circuits

Aditeya Baral¹

¹New York University

1. Motivation

Small transformers trained on arithmetic tasks have been shown to develop clean, interpretable internal algorithms after a grokking phase transition [Power et al., 2022, Nanda et al., 2023]. A fundamental question that this body of work has not addressed is: *does the learned algorithm operate on abstract numerical magnitude, or is it tied to the specific surface representation of numbers used during training?*

The integer five can be written as 5 (decimal), 101 (binary), 5 (octal, coincidentally the same digit), or 5 (hexadecimal, also the same digit) — but for addition problems involving carries, the surface form determines the token sequence in a way that is highly non-trivial. A model trained on binary addition $1101 + 1011 = 11000$ and decimal addition $13 + 11 = 24$ is computing the same abstract operation, but the carry structure, the token sequence length, and the digit vocabulary are all different.

Prior mechanistic interpretability work on arithmetic has studied a single numeral system at a time [Nanda et al., 2023, Kantamneni and Tegmark, 2025, Quirke and Barez, 2024]. It therefore cannot distinguish between a model that has learned an abstract positional-arithmetic algorithm and one that has learned a surface-specific algorithm that happens to generalize within a base. We address this gap directly. We train small transformers from scratch on addition in multiple numeral bases simultaneously, with a shared character-level tokenizer, and apply causal circuit analysis to determine whether the same internal components are responsible for addition across all bases.

2. Significance

This work provides the first causally grounded account of whether arithmetic circuits in transformers are tied to surface representation or encode a genuinely base-agnostic algorithm. By training on multiple bases with a balanced corpus, we eliminate data imbalance as a confound. The four bases chosen — binary, octal, decimal, and hexadecimal — form a natural gradient of token overlap with each other, enabling a structured test of whether circuit sharing correlates with surface similarity. If shared circuits emerge despite large differences in token vocabulary and carry structure (as between binary and decimal), this constitutes strong evidence that transformers can learn abstract positional arithmetic. If circuits are surface-specific, this places a fundamental limit on the generality of the algorithms identified in prior mechanistic interpretability work.

The methodology provides a reusable framework for studying surface-form invariance in transformer circuits beyond arithmetic.

3. Related Work

Mechanistic interpretability of arithmetic. Nanda et al. [2023] reverse-engineered the circuit for modular arithmetic in small transformers, identifying a Fourier-based algorithm whose emergence coincides with a grokking phase transition. Kantamneni and Tegmark [2025] extended this to integer addition in larger models, showing that numbers are represented as a generalized helix and addition is implemented via a Clock algorithm verified with causal interventions. Quirke and Barez [2024] analyzed addition circuits in GPT-2,

identifying carry-related structure in MLP layers. All of this work studies a single numeral system (decimal), leaving open whether the identified algorithms are base-specific or abstract.

Grokking and algorithmic generalization. Power et al. [2022] first characterized grokking — the delayed generalization phenomenon in which a model first memorizes training data and only later transitions to an algorithm that generalizes. Nanda et al. [2023] showed that grokked models develop interpretable circuits, making them strongly preferred for mechanistic analysis. Weight decay is the primary driver of grokking [Power et al., 2022]: it penalizes large weights and pushes the model toward compact, weight-efficient solutions.

Shared circuits across tasks. Recent work has shown that models trained on related tasks can develop shared circuits. Quirke and Barez [2024] found shared carry and borrow circuits in models trained on both addition and subtraction. ? found that subject-verb agreement circuits in English and Spanish are nearly identical and causally transferable. No prior work has studied circuit sharing across *surface representations of numbers*.

Gap addressed by this work. No prior work has applied causal circuit analysis to arithmetic across multiple numeral bases, and no prior work has asked whether the algorithms identified in the grokking literature are surface-specific or genuinely abstract. This is the gap we address.

4. Research Questions

- RQ1:** Does a single small transformer reliably grok addition across all four bases simultaneously when trained on a balanced corpus?
- RQ2:** Which attention heads and MLP layers contribute most to the correct-answer logit for each base, as measured by direct logit attribution?
- RQ3:** Are the components identified in RQ2 consistent across bases — i.e., does the model use the same circuit for addition regardless of the numeral base?
- RQ4:** At which layer does the model’s intermediate representation of operand values converge to a shared encoding, if at all?
- RQ5:** Can activations from one base’s forward pass causally substitute for another base’s activations at shared components, without degrading accuracy?

5. Experimental Setup

Numeral Bases

We study four positional numeral bases chosen for their gradient of token overlap:

Base	Name	Digits	Token overlap with base 10	Example: $13 + 11 = 24$
2	Binary	0, 1	Minimal (2 shared digits)	$1101 + 1011 = 11000$
8	Octal	0–7	High (8 shared digits)	$15 + 13 = 30$
10	Decimal	0–9	— (anchor)	$13 + 11 = 24$
16	Hexadecimal	0–9, A–F	Partial (10 shared + 6 new)	$D + B = 18$

This gradient is central to the experimental design. If circuit sharing correlates with token overlap, the model’s internal algorithm is likely driven by shared surface tokens rather than abstract magnitude. If circuit sharing is constant across the gradient — strong between binary and decimal despite minimal token overlap — this is strong evidence for a genuinely base-agnostic algorithm.

Octal provides an additional sharp test: $15 + 13 = 30$ in base 8, meaning the same underlying computation as $13 + 11 = 24$ in decimal produces entirely different tokens and a carry at a different position. This creates “same computation, different surface” pairs that are useful for activation patching.

Hexadecimal introduces six new digit tokens (A-F) not present in any other base, testing whether the circuit generalizes to novel symbols that have no analog in the model’s other training data.

Prompt Format

Each prompt is formatted as $k = 8$ few-shot examples followed by a test problem:

$$x_1 + y_1 = x_1 + y_1, \dots, x_8 + y_8 = x_8 + y_8, x + y =$$

where all operands and answers are expressed in the same base throughout the prompt. No explicit base-indicator token is included. The model must infer the base solely from the digit characters present in the few-shot examples — a model that succeeds has necessarily learned to condition its arithmetic on the surface representation of numbers.

The few-shot examples for each prompt are drawn randomly from the training operand partition, subject to the constraint that at least two of the eight examples involve a carry, ensuring the in-context demonstrations adequately represent the carry structure of the base. The test problem operands are drawn from the held-out partition, preventing memorization of specific input-output pairs.

For circuit analysis, the few-shot context introduces a confound: activations at the $=$ position are a function of both the test problem and the eight preceding examples. To isolate the contribution of the test problem, we fix the eight few-shot examples to a canonical set per base during evaluation, varying only the test operands. This ensures that differences in activations across prompts reflect the test problem alone, making the LASSO regression and ablation results interpretable as properties of the test computation rather than of the context.

Training Data

We construct a synthetic corpus of addition problems across all four bases. Operands are integers drawn uniformly from $[0, 99]$ in decimal — equivalently, from $[0, 1100011]$ in binary, $[0, 143]$ in octal, and $[0, 63]$ in hexadecimal. We fix the integer range rather than the digit count per base: the resulting variation in sequence length across bases (operands span at most 7 binary digits, 3 octal digits, 2 decimal digits, or 2 hexadecimal digits) is intentional. It reflects the genuine difference in surface representation between bases, and any shared circuit must be robust to it. Sums reach at most 198, requiring at most 8 binary digits, 3 octal digits, 3 decimal digits, or 2 hexadecimal digits. Carry problems — those for which the carry propagates at least one position — are oversampled to ensure the model cannot avoid learning carry structure.

We use a character-level tokenizer with one token per character in the vocabulary $\{0-9, A-F, +, =, ,, \text{space}\}$ (21 tokens total, adding the comma delimiter used between few-shot examples).

Prompts are distributed equally across all four bases at the problem level. All bases see identical operand distributions. A held-out operand split per base prevents memorization of specific input-output pairs.

Model Selection

We conduct a grid search over hidden dimension $d \in \{32, 64, 128, 256\}$, number of layers $L \in \{1, 2, 3\}$, and number of attention heads $H \in \{1, 2, 4, 8\}$, with MLP intermediate dimension fixed at $4d$. All models are trained with AdamW under a cosine learning rate schedule with high weight decay to reliably induce grokking.

We require grokking to occur for each base independently: validation accuracy must saturate strictly after training accuracy for each of the four bases. We select the smallest model by parameter count satisfying this criterion across all four bases simultaneously, with at least 95% exact-match accuracy on the held-out set per base. The minimum-over-bases criterion ensures genuine multi-base generalization rather than average performance.

6. Methodology

Our analysis follows six steps. Step 0 defines the units of analysis and collects the activations that all subsequent steps operate on. Steps 1 and 2 identify and causally confirm the circuit: LASSO proposes a sparse candidate set, and ablation establishes which candidates are causally necessary. Steps 3–5 characterize the confirmed circuit: the logit lens traces how representations evolve across layers, DLA attributes signed contributions to individual components, and activation patching tests whether shared components encode genuinely base-agnostic information.

Step 0: Activation Extraction

Definition of neuron. We use the term *neuron* to refer to either an individual scalar pre-activation in the MLP intermediate layer ($4d$ per layer) or a full attention head output vector treated as a single unit (H per layer). Treating the attention head as an atomic unit is consistent with prior circuit-level analyses, where heads are the natural unit of attention computation. Together, each layer exposes $4d + H$ neurons under this definition.

Token positions. Each prompt consists of eight few-shot examples followed by the test problem. We extract activations at five semantically meaningful token positions within the test problem: the final character of operand a , the final character of operand b , the operator token (+), the = token (the last input token before generation begins), and the first generated answer token. Because the few-shot context is fixed across evaluation prompts (see Prompt Format), activations at these positions vary only with the test operands, making them clean targets for LASSO and ablation. Steps 1–5 focus on the = position, which aggregates the full problem context — including the few-shot demonstrations — immediately prior to generation.

Activation types. At each position and each layer we extract three activation types:

- **Residual stream** ($\mathbb{R}^d$): the accumulated representation at the output of each transformer block.
- **MLP intermediate** ($\mathbb{R}^{4d}$): the scalar pre-activations entering the down-projection of the MLP sub-layer.
- **Attention heads** ($\mathbb{R}^{H \times d_h}$): the output of each attention head prior to the output projection.

We collect activations over 10,000 evaluation prompts per base, yielding 40,000 prompts in total. This dataset of activations is the shared input to all subsequent steps.

Step 1: Identify Candidate Neurons via LASSO

We fit an ℓ_1 -regularized linear regression for each base, predicting the correct answer token’s logit from the neuron activations at the = position. The feature vector comprises, for each layer, the $4d$ MLP intermediate scalars and the ℓ_2 -norm of each attention head output — one scalar per head — concatenated across all L layers into a vector of dimension $m = L(4d + H)$. Formally, let $\mathbf{A} \in \mathbb{R}^{n \times m}$ be the feature matrix over n prompts and $\mathbf{y} \in \mathbb{R}^n$ the correct-answer logit values; we solve:

$$\hat{\beta} = \arg \min_{\beta} \frac{1}{2n} \|\mathbf{A}\beta - \mathbf{y}\|_2^2 + \lambda \|\beta\|_1$$

with λ selected via 5-fold cross-validation. The support of $\hat{\beta}$ constitutes the candidate circuit $\hat{\mathcal{N}}$ for that base. Neurons appearing in $\hat{\mathcal{N}}$ for all four bases are promoted to cross-base candidates and carried forward to Step 2.

Step 2: Confirm Causal Necessity via Ablation

LASSO identifies neurons that are linearly predictive of the correct answer but not necessarily causally necessary. For each neuron in the cross-base candidate set $\hat{\mathcal{N}}$, we zero out its activation at the = position during the forward pass and measure the resulting drop in exact-match accuracy, independently per base. Neurons are classified as:

- **Universally shared:** ablation degrades accuracy in all four bases; the neuron participates in a cross-base circuit.
- **Base-specific:** ablation degrades accuracy in only one or two bases; the neuron is dedicated to surface-specific processing.
- **Non-causal:** ablation has no significant effect; the neuron was predictive but not causally necessary.

The universally shared neurons that survive this filter constitute the causally confirmed circuit $\mathcal{N}^* \subseteq \hat{\mathcal{N}}$. Steps 3–5 characterize $\mathcal{N}^*$.

Step 3: Trace Layer-wise Evolution via Logit Lens

We apply the logit lens [nostalgebraist, 2020] to the confirmed circuit components, projecting the residual stream at each layer onto the output vocabulary and decoding the top token. For each base, we track: (a) whether the decodable token corresponds to the correct answer, and (b) whether it is expressed in the input base or a different one.

If intermediate representations decode as the correct digit in a *different* base than the input — e.g., a binary-input problem whose middle-layer representation decodes as a decimal numeral — this constitutes evidence of an internal base-neutral encoding, analogous to the English-pivot behavior observed in multilingual models [Wendler et al., 2024]. This step localizes the layer at which base-specific surface representations give way to a shared numerical encoding within $\mathcal{N}^*$.

Step 4: Attribute Contributions via Direct Logit Attribution

We apply direct logit attribution (DLA) to the confirmed circuit $\mathcal{N}^*$ to decompose each component’s signed contribution to the correct-answer logit. Because the residual stream is a sum of component outputs, the logit for any token can be written as an exact linear sum: for each attention head and MLP sublayer in $\mathcal{N}^*$, we project its output through the unembedding matrix to obtain a signed scalar.

Formally, let $\mathbf{r}_\ell \in \mathbb{R}^d$ be the residual stream at layer ℓ and $W_U \in \mathbb{R}^{|\mathcal{V}| \times d}$ the unembedding matrix. The contribution of component c at layer ℓ to the logit of token t is:

$$\text{DLA}(c, \ell, t) = W_U[t, :] \cdot \Delta \mathbf{r}_{c, \ell}$$

where $\Delta \mathbf{r}_{c, \ell}$ is the output of component c at layer ℓ . We compute DLA per (base, problem) across all four bases, identifying which components within $\mathcal{N}^*$ contribute most uniformly to the correct-answer logit regardless of base. This provides a fine-grained attribution profile of the shared circuit.

Step 5: Test Representational Sharing via Activation Patching

Ablation establishes which neurons are shared; activation patching establishes *what they encode*. We construct matched problem pairs: two problems representing the same underlying addition (same integer operands and answer) expressed in two different bases. For each component in $\mathcal{N}^*$, we replace its activation from the base- A forward pass into the base- B forward pass and measure whether the correct base- B answer is still produced. We run this for all six base-pair combinations and at each layer to localize where base-agnostic representations emerge.

The octal test case provides a particularly sharp version of this experiment: since $15 + 13 = 30$ in base 8 but $13 + 11 = 24$ in base 10, patching activations from the decimal forward pass into the octal forward pass tests whether shared components encode the correct *base-conditioned* answer or merely the underlying integer sum. The distinction reveals whether the representation is truly abstract or magnitude-encoding only.

7. Expected Outcomes

The six-step pipeline is informative regardless of outcome. We anticipate one of three primary findings:

Shared circuit. The same small set of attention heads and MLP neurons drives addition in all four bases. DLA scores are consistent across bases, ablation produces uniform accuracy drops, and activation patching succeeds across base pairs. This would mean the model learned a genuinely base-agnostic addition algorithm — the carry structure is shared even when the surface tokens differ.

Partially shared circuit. Middle-layer components are shared (magnitude and carry encoding) while early and late components are base-specific (digit embedding and answer decoding). This would localize precisely where the abstraction over surface form occurs, analogous to the language-peripheral, content-central structure observed in multilingual models.

Base-specific circuits. Separate subnetworks develop for each base, with different components responsible for each. Activation patching would fail. This would mean the algorithms identified in prior single-base grokking studies are surface-specific, placing a fundamental limit on their generality.

The gradient structure of the four bases provides a secondary test: if circuit sharing correlates with token overlap (binary least shared, octal most shared relative to decimal), this suggests the sharing is driven by co-occurrence of surface tokens rather than abstract numerical structure.

8. Further Experiments

The primary study establishes *which* components form the shared circuit and *whether* it generalizes across bases. The following extensions pursue two deeper goals: understanding the *mechanism* the shared circuit implements, and testing the *boundaries* of that mechanism under more demanding conditions.

Recovering the underlying algorithm. Identifying the confirmed circuit $\mathcal{N}^*$ tells us which components are necessary but not what algorithm they implement. Prior work on single-base arithmetic has characterized specific algorithmic structures: Nanda et al. [2023] found that grokked transformers implement modular addition via a Fourier basis in weight space, and Kantamneni and Tegmark [2025] found that integer addition is implemented via a Clock algorithm in which numbers are represented as points on a generalized helix. The central question this extension asks is: does the same algorithm appear across all four bases within $\mathcal{N}^*$, or do bases that share a circuit nonetheless implement it via structurally different internal representations?

Concretely, we will analyze the weight matrices of components in $\mathcal{N}^*$ for Fourier structure, visualize the geometry of operand representations (testing for helical structure as in Kantamneni and Tegmark [2025]), and apply Discrete Fourier Transforms to MLP neuron activations as a function of operand value. If the same frequencies dominate across all four bases, the model has not only shared the circuit but implemented a single base-agnostic algorithm within it. If different frequencies or geometric structures appear per base despite shared components, the circuit sharing is structural but the algorithm is base-specific — a subtler and more interesting finding.

Subtraction. Subtraction is the most natural extension because borrowing is structurally carry run in reverse. The cross-base question applies directly: does the model develop a shared borrow circuit across all four bases? The more interesting secondary question is whether the carry and borrow circuits are *unified* — the same components distinguished by a sign or direction signal — or *separate but each base-agnostic*. A unified circuit would constitute stronger evidence for abstract positional-arithmetic computation. Mechanistic analysis of the borrow circuit via the same six-step pipeline would further test whether the Fourier or helix structure found for addition also underlies subtraction, or whether a qualitatively different algorithm emerges.

Roman numerals as a non-positional boundary. Roman numerals (e.g., III + V = VIII) represent the sharpest possible test of the mechanism’s limits: they are non-positional, so the carry-based algorithm identified in the primary study cannot apply. If the LASSO and ablation pipeline identifies any overlap between the Roman numeral circuit and $\mathcal{N}^*$, this would indicate a more abstract magnitude-level representation than carry structure alone. If circuits diverge entirely, Roman numerals establish a clean lower bound on what surface properties a shared mechanism requires.

Operand range scaling. The primary study uses operands from $[0, 99]$ in decimal, giving sums of at most 198. Scaling to $[0, 999]$ adds one decimal digit but roughly three binary digits, creating increasingly unequal sequence lengths across bases. Whether the confirmed circuit $\mathcal{N}^*$ survives this asymmetry — and whether the Fourier or helix structure remains intact at larger operand magnitudes — tests how deeply the model has abstracted over both surface form and problem scale.

References

- Subhash Kantamneni and Max Tegmark. Language models use trigonometry to do addition, 2025. URL <https://arxiv.org/abs/2502.00873>.
- Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=9XFSbDZno3>.
- nostalgebraist. Interpreting GPT: The logit lens. LessWrong, 2020. URL <https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens>.
- Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv preprint arXiv:2201.02177*, 2022. URL <https://arxiv.org/abs/2201.02177>.
- Philip Quirke and Fazl Barez. Understanding addition in transformers. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=rrjlxR3tOp>.
- Chris Wendler, Veniamin Veselovsky, Giovanni Monea, and Robert West. Do Llamas work in English? On the latent language of multilingual transformers. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15366–15394, Bangkok, Thailand, 2024. Association for Computational Linguistics. URL <https://aclanthology.org/2024.acl-long.820>.